



Habib University

CE 321/CS 330 Computer Architecture – Fall 2025

Instructor: Dr. Shehzad Hasan

Time = 40 minutes

Quiz 03

Max Points: 20

Name: _____

ID: _____

Instructions:

- i. Smart watches, laptops, and similar electronics are strictly NOT allowed.
- ii. Answer sheets should contain all steps, working, explanations, and assumptions.
- iii. Attempt the quiz on clean papers with black/blue ink.
- iv. Print your name and HU ID on all sheets.
- v. Turn in your question paper along with your answer sheets.
- vi. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	✓
CLO 4	<i>Analyze the</i> performance of cache operations	

Question 1 [5 marks]:

Let's say we have a 7-stage pipelined processor. Delay (in ps) for each stage is given below

Fetch	Decode	EXE1	EXE2	EXE3	Memory	Write-back
250	200	300	250	250	350	300

Inter-stage pipeline registers take an additional 20 ps delay

- a) What will be clock cycle time for a single-cycle version of this processor?

$$250+200+300+250+250+350+300=1900 \text{ ps}$$

- b) What will be the clock cycle time for a pipelined version of this processor?

$$\max\{T_i\} + T_r = 350 + 20 = 370 \text{ ps}$$

- c) What is the maximum speed-up we can achieve with this pipelined processor?

Max speedup is achieved when n approaches infinity

$$(1900 \times n) / (370 \times 7 + (n-1) 370) = 1900n / (2220 + 370n) = 5.135$$

- d) What is the ideal/theoretical max speed-up we can achieve with a 7-stage pipelined processor, considering no register delays?

The theoretical maximum speedup is equivalent to the number of stages i.e. 7

- e) Are the speed-ups in part c and d same? If yes, why? If not, why?

No, they are different because the maximum speedup is achieved when stages are of equal durations

Question 1 [5 marks]:

Let's say we have a 7-stage pipelined processor. Delay (in ps) for each stage is given below

Fetch	Decode	EXE1	EXE2	EXE3	Memory	Write-back
350	300	400	350	350	450	400

Inter-stage pipeline registers take an additional 40 ps delay

- a) What will be clock cycle time for a single-cycle version of this processor?

$$350+300+400+350+350+450+400=2600 \text{ ps}$$

- b) What will be the clock cycle time for a pipelined version of this processor?

$$\max\{T_i\} + T_r = 450 + 40 = 490 \text{ ps}$$

- c) What is the maximum speed-up we can achieve with this pipelined processor?

Max speedup is achieved when n approaches infinity

$$(2600 \times n) / (490 \times 7 + (n-1) 490) = 2600n / (2220 + 490n) = 5.306$$

- d) Same as before

- e) Same as before

Question 2 [5 marks]:

Assume the following sequence of instructions is executed on the 5-stage RISC-V pipelined processor

```
LOOP:    ld x2, 16(x23)
         ld x1, 8(x23)
         add x3, x1, x2
         addi x23, x23, -24
         beq x3, x0, LOOP
         add x5, x6, x7
         addi x5, x0, 300
```

- a) If there is no forwarding or hazard detection, insert NOPs to ensure correct execution. BTA is calculated in the EX stage. (3 marks)

The compiler would insert nops whenever a dependent instruction couldn't get its right value. In case we have no forwarding paths this means there should be a difference of two cycles between an instruction producing result and some other using it.

```
LOOP:    ld x2, 16(x23)
         ld x1, 8(x23)
         nop
         nop
         add x3, x1, x2
         addi x23, x23, -24
         nop
         beq x3, x0, LOOP
         nop
         nop
         add x5, x6, x7
         addi x5, x0, 300
```

- b) If the processor has forwarding in the EX stage, but no hazard detection unit in the ID stage, what happens when the original code executes? (state which instructions will execute incorrectly) (2 marks)

With no Hazard Detection Unit only load use hazards would be incorrect. So, add x3, x1, x2 will not get the right data and obviously any instruction depending on this will also be incorrect e.g. beq

Question 3 [5 marks]:

Consider the code given below

```

loop: ld    x10, 0(x2)
      add   x1, x0, x10
      ld    x11, 4(x2)
      addi  x2, x2, 4
      sub   x3, x1, x11
      beq   x0, x0, loop
      addi  x6, x0, 100

```

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

- a) Assume that the given code runs on a 5-stage pipelined RISC-V processor which has hazard detection unit and forwarding unit for ALU implemented. For this loop, specify values of FwdA, FwdB, PCWrite, IF/ID_Write and Control MUX signals for each cycle in the table below. For a normal instruction, PCWrite and IF/ID_Write both are 1, whereas Control MUX is 0. For a stalled instruction, PCWrite and IF/ID_Write are 0, whereas Control MUX is 1. Values of FwdA and FwdB are given above. All branch decisions are taken in the ID stage. Branches are predicted to be not-taken.

Instr/cycle	1	2	3	4	5	6	7	8	9	10	11	12	13
ld	IF	ID	EX	M	WB								
add		IF	ID	ID _{again}	EX	M	WB						
ld			IF	IF _{again}	ID	EX	M	WB					
addi					IF	ID	EX	M	WB				
sub						IF	ID	EX	M	WB			
beq							IF	ID	EX	M	WB		
addi								IF	Flush				
ld									IF	ID	EX	M	WB
add										IF	ID	ID _{again}	EX
ld											IF	IF _{again}	ID
PCWrite	-	1	0	1	1	1	1	1	1	1	0	1	1
IF/ID_Write	-	1	0	1	1	1	1	1	1	1	0	1	1
Control MUX	-	0	1	0	0	0	0	0	0	0	1	0	0
FwdA	-	-	00	00	00	00	00	00	00	00	00	00	00
FwdB	-	-	00	00	01	00	x	01	00	00	00	00	01

Question 3 [5 marks]:

Consider the code given below

```

loop: ld    x10, 0(x2)
      add   x1, x0, x10
      ld    x11, 4(x1)
      addi  x2, x2, 4
      sub   x3, x2, x11
      beq   x0, x0, loop
      addi  x6, x0, 100

```

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

- b) Assume that the given code runs on a 5-stage pipelined RISC-V processor which has hazard detection unit and forwarding unit for ALU implemented. For this loop, specify values of FwdA, FwdB, PCWrite, IF/ID_Write and Control MUX signals for each cycle in the table below. For a normal instruction, PCWrite and IF/ID_Write both are 1, whereas Control MUX is 0. For a stalled instruction, PCWrite and IF/ID_Write are 0, whereas Control MUX is 1. Values of FwdA and FwdB are given above. All branch decisions are taken in the ID stage. Branches are predicted to be not-taken.

Instr/cycle	1	2	3	4	5	6	7	8	9	10	11	12	13
ld	IF	ID	EX	M	WB								
add		IF	ID	ID _{again}	EX	M	WB						
ld			IF	IF _{again}	ID	EX	M	WB					
addi					IF	ID	EX	M	WB				
sub						IF	ID	EX	M	WB			
beq							IF	ID	EX	M	WB		
addi								IF	flush				
ld									IF	ID	EX	M	WB
add										IF	ID	ID _{again}	EX
ld											IF	IF _{again}	ID
PCWrite	-	1	0	1	1	1	1	1	1	1	0	1	1
IF/ID_Write	-	1	0	1	1	1	1	1	1	1	0	1	1
Control MUX	-	0	1	0	0	0	0	0	0	0	1	0	0
FwdA	-	-	00	00	00	10	00	10	00	00	00	00	00
FwdB	-	-	00	00	01	00	x	01	00	00	00	00	01

Question 4 [5 marks]:

a) Write a set of conditions using proper notations for detecting a load-use hazard in the ID stage. (2 marks)

if {

ID/EX.MemRead &&

(ID/EX.Register_rd == IF/ID.Register_rs1) || (ID/EX.Register_rd == IF/ID.Register_rs2) &&

(ID/EX.Register_rd != 0)

}

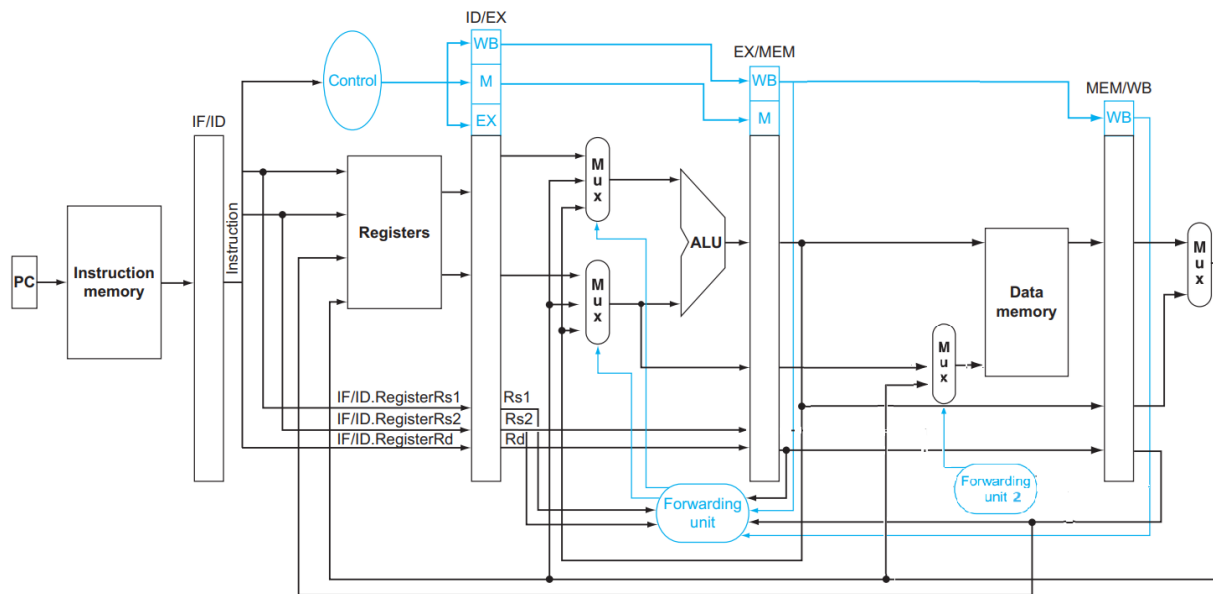
then stall

b) A sequence of instructions as shown below can result in data hazards that need stall cycles.

ld x20, 0(x5)

sd x20, 100(x5)

However, this can be resolved using another forwarding unit Forwarding Unit - 2 in the MEM stage, which passes on the right value to the memory. This new path would be from the MEM/WB pipeline register back to Data Memory. Write the expression for detecting this sequence and make changes in the diagram by showing all the required connections to Forwarding Unit - 2. (3 marks)



Expression

if {

(EX/MEM.MemWrite) &&

(MEM/WB.Register_rd == EX/MEM.Register_rs2) &&

(MEM/WB.Register_rd != 0) && (MEM/WB.RegWrite)

}